Lecture01

1.

Cell-based ASIC

- ✓ **Cell-based IC (CBIC)**
  - use ***pre-designed*** logic cells (known as standard cells) and micro cells (e.g. microcontroller)
  - designers save time, money, and reduce risk
  - each standard cell can be optimized individually
  - all mask layers are customized
  - custom blocks can be embedded

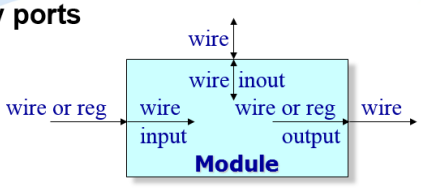
Full-Custom Design

- ✓ **An engineer designs some or all of the logic cells, circuits, layout specifically for one ASIC**
  - required cells/IPs are not available
  - existing cell libraries are not fast enough
  - logic cells are not small enough or consume too much power
  - technology migration (mixed-mode design)
  - ***demand long design cycle***
- ✓ **Not our focus**

2.

Port

- ✓ **Interface is defined by ports**
- ✓ **Port declaration**
  - input : input port
  - output : output port
  - inout : bidirectional port



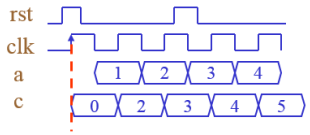
- ✓ **Port connection**
  - input : only wire can be assigned to represent this port in the module
  - output : only wire can be assigned to represent this port out of module
  - inout : register assignment is forbidden neither in module nor out of module ***[Tri-state]***

1.

Flip-Flop Designs

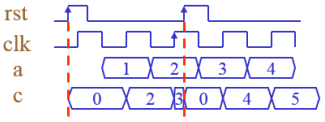
- **Register with synchronous reset**
  - Syntax: @(posedge clk) : for synchronous reset

```
EX.
always @(posedge clk)
begin
  if(rst)     c <= 0;
  else        c <= a + 1;
end
```



- **Register with asynchronous reset**
  - Syntax: @(posedge clk or posedge rst)

```
EX.
always @(posedge clk or posedge rst)
begin
  if(rst)     c <= 0;
  else        c <= a + 1;
end
```



Flip-Flop Designs – Synchronous Resets (cont.)

- **Advantages**
  - Easier work with cycle based simulator
  - Typical recommended for DFT design
    - DFT (Design for test)
  - Easier for ECO (Engineering Change Order)
  - Glitch filtering from reset combinational logic
    - Filter the small glitch of reset between clock
  - Glitch filtering if reset is in a mission-critical application
    - When reset is generated by a set of internal condition, it can filter the glitch of logic equations between clock.
- **Disadvantages**
  - May not be able come out unknown X during simulation
  - Adds delay to data path
  - Can't be reset without clock signal

Flip-Flop Designs – Asynchronous Resets (cont.)

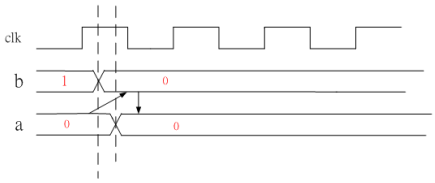
- ✓ **Advantages**
  - Reset is immediate
  - No problem related to unknown X propagation in simulation
  - Does not interfere or add extra delay to data path
  - Very easy to implement
  - Reset is independent of clock signal
- ✓ **Disadvantages**
  - Noisy reset line could cause unwanted reset
    - This can be filtered
  - Difficult for ECO
  - DFT prefer synchronous designs

2.

Assignments – Procedural Assignment (cont.)

- ✓ **Blocking procedural assignment**
  - Syntax: <value> = <timing_control><expression>
  - Must be executed before executing the statement that follow it in a sequential block.
    - Example

```
initial begin
  a=0;b=1;clk=0;
end
always @(posedge clk)
begin
  b = #1 a;
  a = #1 b;
end
```

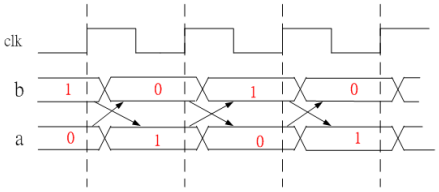


Assignments – Procedural Assignment (cont.)

✓ Non-Blocking procedural assignment

- Syntax: <value> <= <timing_control><expression>
- The statements are executed **within the same time step** without regard to order or dependence upon each other.
- Evaluating Non-blocking divided into two steps:
  - Example

```
initial begin
   a=0;b=1;clk=0;
end
always@(posedge clk)
 begin
     b <= #1 a;
     a <= #1 b;
 end
```



- ◆ Step1 : at posedge clk the simulator evaluates the RHS's new value, and scheduled when to update LHS.
- ◆ Step2 : at 1ns after posedge clk the simulator updates LHS. Because the given delay (#1) is expired.

Lecture03

1.

```
# 1ns :$display: a=0 b=1
# 1ns :#0      : a=0 b=1
# 1ns :$monitor: a=1 b=0
# 1ns :$strobe : a=1 b=0
#
# 2ns :$display: a=1 b=0
# 2ns :#0      : a=1 b=0
# 2ns :$monitor: a=1 b=0
# 2ns :$strobe : a=1 b=0
```

2.

1. In module DESIGN(…), the variable “out” should be declared as “wire ” instead of “reg”.
2. In module DESIGN(…), the variable “2_1”violates the naming rule, it should begin with _ or alphabets, such as “w2_1”.
3. In module DESIGN(…), task can’t be synthesizable. So can not be use in design.
4. In module PATTERN (…), there’s no finish in pattern. The simulation will keep running.
5. In dumping the waveform in TESTBED, it need to add the cmd “$sdf_annotate("DESIGN _SYN.sdf", My_ DESIGN);” in gate simulation.
6. In TESTBED, when instantiating DESIGN and PATTERN modules, a name should be given, such as DESIGN mux_1(.clk(clk),….);

…..(more than 6)

Lecture04

1.

Ans:

Operator inference:

- ✓ Use the HDL operator in description
- ✓ Convenient
- ✓ Sometimes it is inefficient when synthesizing
- ✓ Supply default function only, cannot use special function.

Instantiate IP:

- ✓ To instantiate a synthetic module manually and explicitly
- ✓ Need to include a reference to the synthetic module in HDL code.
- ✓ Supply different architecture for implementation
- ✓ Apply pre-compiling sub-blocks speeds up the synthesis for large design.

2.

(a) Propagation delay is the maximum amount of time from an input changes until all outputs reach steady state. Clk-to-Q is the propagation delay in flip-flop, and logic propagation delay is the propagation delay in combinational logic.

(b)

	Setup time	Hold time
Data required time =	$t_{cycle} - t_{setup}$	t_{hold}
Data arrival time =	$t_{pcq} + t_{pd}$	t_{ccq}+t_{cd}
Criterion :	$(t_{cycle} - t_{setup}) > (t_{pcq} + t_{pd})$	$(t_{ccq}$+$t_{cd}) > (t_{hold})$

Lecture 05

1. When using memory, we use memory compiler to generate two files used in our design flow, v file and lib file. Separately explain what are this two files and which part of design flow are they used in.

v file : Behavior model which cannot be synthesized. Use in rtl simulation and gate level simulation.

lib file : Library for synthesis & APR. It gives information such as delay, timing constraint, area, capacitance of ports of the memories, that enable the synthesizer to perform static timing analysis (STA) based on these data. However unlike designware, the memory will not be synthesized out. Since memory is a hard macro, a compacted layout architecture which is not built from the standard cell gate. Only the timing / area and other information are used for synthesis.

2. Why using sequential logic to describe control signals of memory such as WEN, A, D, ..., or else will be easier to face hold time violation in gate level simulation.

   With sequential logic, Flip-Flop output is connected to the Memory input. If the clk->q delay is smaller than the hold time check of memory, a hold time violation will occur.

Lecture06

Synthetic_library:

Specifies additional **DesignWare libraries** for optimization purposes.

Efficient implementations for adders, comparators, multipliers

Link_library:

Specifies a list of libraries that Design Compiler can use to resolve design references.

Target_library:

During synthesis, compiler selects gates from target library(usually put **technology library** here).

It also calculates the timing of the circuit, using the vendor-supplied (UMC, TSMC…) timing data of the .lib or .db file.

Top-down compile, in which the top-level design and all its subdesigns are compiled together

Bottom-up compile, in which the individual subdesigns are compiled separately, starting from the bottom of the hierarchy and proceeding up through the levels of the hierarchy until the top-level design is compiled.

Top-down compile achieve less synthesis time but Bottom-up compile achieve better performance